

CSA0613 - Design and Analysis of Algorithms

CO5-AT1-Constraint-Based Problem Solving

Divyesh S P (192424147).

Graph Isomorphism Verification Using Backtracking

1. Aim

To implement a **backtracking-based algorithm** to determine whether two graphs are isomorphic by checking vertex mapping and edge preservation constraints.

2. Problem Statement

Given two graphs, determine whether one graph can be transformed into the other by **relabeling its vertices** while preserving the connections between vertices.

3. Methodology

- Represent both graphs using adjacency matrices.
- Check whether both graphs have the same number of vertices.
- Compare the degree sequences of the graphs.
- Generate possible vertex mappings using backtracking.
- Check whether the mapping preserves all existing edges.
- Reject invalid mappings early using constraint checking.
- If a valid mapping is found, declare the graphs isomorphic.

4. Constraints

1. Both graphs must have the same number of vertices.
2. Each vertex must map to only one unique vertex.
3. Corresponding vertices must have the same degree.

4. Edges must be preserved after mapping.

5. Implementation

```
def is_isomorphic(G1, G2):
    n = len(G1)

    if len(G2) != n:
        return False

    degree1 = [sum(row) for row in G1]
    degree2 = [sum(row) for row in G2]

    if sorted(degree1) != sorted(degree2):
        return False

    mapping = [-1] * n
    used = [False] * n

    def backtrack(v):
        if v == n:
            return True

        for u in range(n):
            if not used[u] and degree1[v] == degree2[u]:

                valid = True

                for prev in range(v):
                    if mapping[prev] != -1:
                        if G1[v][prev] != G2[u][mapping[prev]]:
                            valid = False
                            break

                if valid:
                    mapping[v] = u
                    used[u] = True
```

```

        if backtrack(v + 1):
            return True

        mapping[v] = -1
        used[u] = False

    return False

return backtrack(0)

G1 = [
    [0, 1, 1, 0],
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [0, 0, 1, 0]
]

G2 = [
    [0, 1, 0, 1],
    [1, 0, 1, 1],
    [0, 1, 0, 0],
    [1, 1, 0, 0]
]

if is_isomorphic(G1, G2):
    print("The graphs are ISOMORPHIC.")
else:
    print("The graphs are NOT ISOMORPHIC.")

```

6. Input

Graph 1

```

0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0

```

Graph 2

0 1 0 1
1 0 1 1
0 1 0 0
1 1 0 0

7. Output

```
# Graph 1
G1 = [
    [0, 1, 1, 0],
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [0, 0, 1, 0]
]

# Graph 2 - isomorphic to Graph 1
G2 = [
    [0, 1, 0, 1],
    [1, 0, 1, 1],
    [0, 1, 0, 0],
    [1, 1, 0, 0]
]

if is_isomorphic(G1, G2):
    print("The graphs are ISOMORPHIC.")
else:
    print("The graphs are NOT ISOMORPHIC.")
```

... The graphs are ISOMORPHIC.

8. Non-Isomorphic Test Case

Use the following Graph 2:

0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0

Output

```
# Graph 1
G1 = [
    [0, 1, 1, 0],
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [0, 0, 1, 0]
]

# Graph 2
G2 = [
    [0, 1, 1, 0],
    [1, 0, 0, 1],
    [1, 0, 0, 1],
    [0, 1, 1, 0]
]

if is_isomorphic(G1, G2):
    print("The graphs are ISOMORPHIC.")
else:
    print("The graphs are NOT ISOMORPHIC.")
```

... The graphs are NOT ISOMORPHIC.

9. Complexity Analysis

Time Complexity:

Worst case: $O(n! \times n^2)$

The algorithm may need to explore many possible vertex mappings. Constraint checking reduces unnecessary mappings through early pruning.

Space Complexity:

$O(n)$ additional space for mapping and visited arrays.

10. Correctness

The algorithm considers possible one-to-one vertex mappings and accepts a mapping only when the degree and edge-preservation constraints are satisfied. Therefore, if the algorithm returns **ISOMORPHIC**, a valid vertex correspondence exists between the two graphs.

11. Result

The **Graph Isomorphism Verification** problem was successfully implemented using **backtracking and constraint checking**. The program correctly identifies both isomorphic and non-isomorphic graphs. Early constraint checking helps eliminate invalid vertex mappings before exploring complete mappings.